TECHNICAL REFERENCE

Validating RAG Systems

Evaluation concepts, the RAGAS & DeepEval vocabularies, when to use each, and worked code examples.

Prepared June 19, 2026
Reflects RAGAS v0.2+ and current DeepEval APIs

1 Foundations of RAG Evaluation

A RAG pipeline has two parts that can fail independently, and they should be measured separately before judging the pipeline as a whole.

- **Retrieval** — did the system pull the right context?
- **Generation** — given that context, did the model produce a good, grounded answer?

Looking only at the final answer hides *where* a failure came from: bad output could stem from retrieving the wrong chunks or from the model mishandling good chunks. Diagnose each layer.

Build an evaluation set first

You need representative queries paired with (a) a reference / "golden" answer and (b) the source passages that actually contain the answer. Curate by hand, or generate synthetically and then have a human review. Without ground truth you are guessing.

Retrieval metrics

- **Recall@k** — was the correct passage in the top `k`? Usually the most important: if the answer is not retrieved, generation cannot succeed.
- **Precision@k** — how much of what was retrieved was actually relevant.
- **MRR / NDCG** — reward ranking the right chunk higher.

Generation metrics (the "RAG triad")

- **Faithfulness / groundedness** — does every claim trace back to the retrieved context, or did the model hallucinate?
- **Answer relevance** — does the answer address the question asked?
- **Correctness** — does it match the reference answer?

Faithfulness and relevance resist exact-match scoring, so the common approach is **LLM-as-judge**: a strong model checks whether each claim is supported by the context. Validate the judge against human ratings on a sample.

> **RUN IT AS A REGRESSION SUITE**
> Every time you change chunking, the embedding model, the retriever, or the prompt, re-run the eval set and watch the metrics. That is how you tell whether an "improvement" actually helped. Keep periodic human spot-checks for high-stakes cases — judges miss subtle contradictions.

2 RAGAS Terminology

RAGAS organizes its vocabulary into **data fields** (what you feed each metric) and the **metrics** themselves.

The data fields (a "sample")

- `user_input` — the question (older name: `question`).

- `retrieved_contexts` — chunks the retriever pulled (older: `contexts`).
- `response` — the generated answer (older: `answer`).
- `reference` — the ground-truth answer (older: `ground_truth`).

RAGAS distinguishes **reference-free** from **reference-based** metrics; it was originally designed to lean on LLM judgment rather than human-annotated labels, so many metrics work without ground truth.

Retrieval metrics

- **Context Precision** — are the relevant chunks ranked highly? Variants include `LLMContextPrecisionWithoutReference` and a with-reference version.
- **Context Recall** — did retrieval bring back everything needed to answer?
- **Context Entities Recall** — stricter recall focused on key entities; useful in fact-heavy domains.
- **Noise Sensitivity** — how easily noisy chunks mislead the system into wrong claims (lower is better).

Generation metrics

- **Faithfulness** — factual consistency of the answer against the retrieved context; the core hallucination check.
- **Response Relevancy** — how well the answer addresses the question. *Renamed*: was "Answer Relevancy"; class is now `ResponseRelevancy`.

Comparison & other metrics

- **Factual Correctness** — fact-level overlap between response and reference.
- **Semantic Similarity** — embedding similarity between response and reference.
- Traditional scorers also exposed: BLEU, ROUGE, string/exact match.
- **Nvidia set**: Answer Accuracy, Context Relevance, Response Groundedness — cheaper/faster judge alternatives.

> **WATCH THE RENAMES**
> `question` → `user_input`, `answer` → `response`, `ground_truth` → `reference`, `answer_relevancy` → `ResponseRelevancy`. Following an old tutorial against a current install is the usual source of confusion; an official v0.1→v0.2 migration guide lists the changes.

3 DeepEval Terminology

DeepEval (by Confident AI) is structured around **test cases** rather than data columns, and uses "Contextual" where RAGAS says "Context."

The test case (unit of evaluation)

Everything runs against an `LLMTestCase`:

- `input` — the user query.
- `actual_output` — what the pipeline produced.
- `expected_output` — the ideal answer (only reference-based metrics need it).

- `retrieval_context` — chunks the retriever returned.
- `context` — an optional "ideal" context, distinct from what was retrieved.

Multi-turn evaluation uses a separate `ConversationalTestCase`; DeepEval provides multi-turn equivalents of every single-turn RAG metric via a sliding window, catching context drift and cross-turn hallucination.

Retriever metrics

- **Contextual Precision** — are the most relevant chunks ranked first? (RAGAS Context Precision.)
- **Contextual Recall** — was everything needed retrieved? Reference-based (needs `expected_output`).
- **Contextual Relevancy** — fraction of retrieved chunks that are actually relevant.

Generator metrics

- **Faithfulness** — does the output factually align with the retrieval context?
- **Answer Relevancy** — how relevant the answer is to the input.

The **RAG triad** DeepEval markets: answer relevancy, faithfulness, and contextual relevancy — each maps to a tunable knob (chunk size, top-K, etc.).

Custom-metric primitives (the big differentiator)

- **G-Eval** — define a metric in plain-English criteria; DeepEval builds an LLM judge. Best for subjective things (tone, correctness).
- **DAG (Deep Acyclic Graph)** — a decision-tree judge giving *deterministic* scores for objective/mixed criteria.
- **Scorer module** — statistical metrics (ROUGE, BLEU) wrapped into a custom metric.

> **IT ALSO WRAPS RAGAS**
> DeepEval ships a `RagasMetric` that averages four sub-metrics (answer relevancy, faithfulness, contextual recall, contextual precision), each also available individually. DeepEval notes its native metrics expose the judge's reasoning and can JSON-confine any custom LLM — useful because RAGAS can emit `NaN` on invalid JSON.

4 RAGAS vs DeepEval — When to Use Which

Dimension	RAGAS	DeepEval
Core purpose	RAG-focused metric library + synthetic test-set generation	Broad LLM eval/testing framework (RAG is one part)
Mental model	Data fields (`user_input` , `response` …)	Test cases (`LLMTestCase`) you assert against
Built-in scope	RAG metrics, plus newer agent/SQL/NLP metrics	50+ metrics: RAG, agentic, multi-turn, red-teaming
Custom metrics	More limited; largely predefined	Strong — G-Eval (criteria) and DAG (deterministic)
Determinism / debug	Score equations; can hit `NaN` on bad JSON	Judge reasoning exposed; JSON-confinable
CI/CD & testing	More a measurement library	Built for it — pytest-style asserts, `deepeval test run`
Multi-turn RAG	Has multi-turn support	First-class `ConversationalTestCase` for every metric
Synthetic data	Signature strength — knowledge-graph test-set generation	Has generation, but not its headline feature
Ecosystem	Framework-agnostic, open source	Open source, nudges toward Confident AI cloud
Best when…	You want fast standardized RAG scoring and to auto-generate an eval set from your docs	You want unit-test-style gates in CI/CD, custom/subjective judges, or coverage beyond RAG

> **THEY ARE NOT MUTUALLY EXCLUSIVE**
> DeepEval can run RAGAS metrics inside its own harness, so a common pattern is **RAGAS for test-set generation** and **DeepEval as the test runner**.

5 Worked Example: Scoring in Both Frameworks

The same evaluation — one question, with retrieved context and a reference answer — in each framework.

RAGAS

Assemble a dataset of samples, pick metrics, call `evaluate()` , get a score table back.

```python
from ragas import evaluate, EvaluationDataset
from ragas.metrics import (
    Faithfulness, ResponseRelevancy,
    LLMContextPrecisionWithoutReference, LLMContextRecall,
)
from ragas.llms import LangchainLLMWrapper
from langchain_openai import ChatOpenAI

judge = LangchainLLMWrapper(ChatOpenAI(model="gpt-4o", temperature=0))

dataset = EvaluationDataset.from_list([
    {
        "user_input": "When was the Eiffel Tower completed?",
        "retrieved_contexts": [
            "The Eiffel Tower was completed in 1889 for the World's Fair.",
            "It stands 330 metres tall in Paris.",
        ],
        "response": "It was completed in 1889.",
        "reference": "The Eiffel Tower was completed in 1889.",
    },
])

result = evaluate(
    dataset=dataset,
    metrics=[
        LLMContextPrecisionWithoutReference(),  # retriever
        LLMContextRecall(),                      # retriever (uses reference)
        Faithfulness(),                          # generator
        ResponseRelevancy(),                     # generator
    ],
    llm=judge,
)
df = result.to_pandas()  # per-sample breakdown
```

DeepEval

Build test cases, attach metrics with thresholds, and run — the natural form is a pytest-style pass/fail.

```python
from deepeval import evaluate
from deepeval.test_case import LLMTestCase
from deepeval.metrics import (
    AnswerRelevancyMetric, FaithfulnessMetric,
    ContextualPrecisionMetric, ContextualRecallMetric,
)

test_case = LLMTestCase(
    input="When was the Eiffel Tower completed?",
    actual_output="It was completed in 1889.",
    expected_output="The Eiffel Tower was completed in 1889.",
    retrieval_context=[
        "The Eiffel Tower was completed in 1889 for the World's Fair.",
        "It stands 330 metres tall in Paris.",
    ],
)

metrics = [
    ContextualPrecisionMetric(threshold=0.7, model="gpt-4o"),  # retriever
    ContextualRecallMetric(threshold=0.7, model="gpt-4o"),     # retriever
    FaithfulnessMetric(threshold=0.7, model="gpt-4o"),         # generator
    AnswerRelevancyMetric(threshold=0.7, model="gpt-4o"),      # generator
]

evaluate(test_cases=[test_case], metrics=metrics)
```

Turn it into a CI/CD gate (save as `test_rag.py`, run `deepeval test run test_rag.py`):

```python
from deepeval import assert_test
from deepeval.test_case import LLMTestCase
from deepeval.metrics import FaithfulnessMetric, AnswerRelevancyMetric

def test_rag_quality():
    case = LLMTestCase(
        input="When was the Eiffel Tower completed?",
        actual_output="It was completed in 1889.",
        retrieval_context=["The Eiffel Tower was completed in 1889..."],
    )
    assert_test(case, [FaithfulnessMetric(threshold=0.7),
                       AnswerRelevancyMetric(threshold=0.7)])
```

Key workflow differences

- **Field names differ.** RAGAS `user_input / retrieved_contexts / response / reference` vs DeepEval `input / retrieval_context / actual_output / expected_output`.
- **Output shape.** RAGAS returns a score table (measurement). DeepEval centers on thresholds and pass/fail (gating).
- **Judge LLM placement.** RAGAS sets one judge per `evaluate()` run; DeepEval sets the model per metric.
- **RAGAS inside DeepEval.** Swap in `from deepeval.metrics.ragas import RagasMetric` to keep RAGAS scoring within DeepEval's harness.

6 Worked Example: Generating a Test Set with RAGAS

This is the part of RAGAS that changed most. The snippet below is the current v0.2+ API.

> **OLD API WILL NOT WORK**
> Older tutorials show `from_langchain(generator_llm, critic_llm, embeddings)` with a `distributions={simple, reasoning, multi_context}` dict. That is v0.1 and fails on current installs.

```python
from langchain_community.document_loaders import DirectoryLoader
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper
from ragas.testset import TestsetGenerator

# 1. Load your own documents
loader = DirectoryLoader("./my_docs", glob="**/*.md")
docs = loader.load()

# 2. Wrap the generator LLM + embedding model
generator_llm = LangchainLLMWrapper(ChatOpenAI(model="gpt-4o"))
generator_embeddings = LangchainEmbeddingsWrapper(OpenAIEmbeddings())

# 3. Build generator and generate
generator = TestsetGenerator(llm=generator_llm, embedding_model=generator_embeddings)
dataset = generator.generate_with_langchain_docs(docs, testset_size=10)

# 4. Inspect / export
df = dataset.to_pandas()
```

What happens under the hood

This is not "ask an LLM for 10 questions." RAGAS ingests documents into a **KnowledgeGraph**, runs transforms to enrich it (summarization, entity extraction, embeddings, relationship-building), generates persona objects for likely users, draws scenarios from the graph according to a query distribution, and synthesizes samples per scenario. The graph step is why it can produce multi-hop questions spanning several chunks.

What you get out

Each sample is a `(user_input, reference_contexts, reference)` triple, and the Testset converts to an `EvaluationDataset` — the columns line up directly with what `evaluate()` expects, so the generated set feeds straight into the metrics from Section 5.

Controlling question types

```
from ragas.testset.synthesizers import default_query_distribution

query_distribution = default_query_distribution(generator_llm)
dataset = generator.generate_with_langchain_docs(
    docs, testset_size=10, query_distribution=query_distribution,
)
```

Two-phase variant (cache the graph)

For larger corpora, build and persist the knowledge graph once, then generate repeatedly without re-processing — the transform phase is the expensive part.

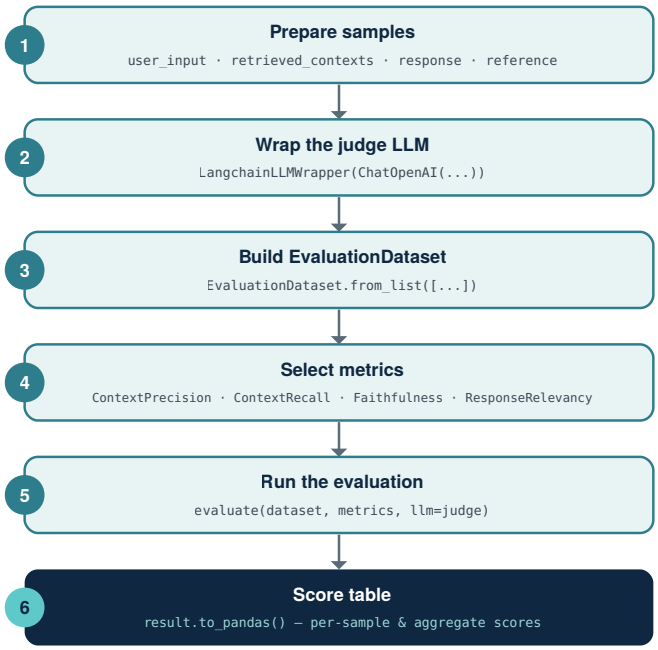
```
from ragas.testset.transforms import default_transforms, apply_transforms
from ragas.testset.graph import KnowledgeGraph

kg = KnowledgeGraph()
trans = default_transforms(documents=docs, llm=generator_llm,
                           embedding_model=generator_embeddings)
apply_transforms(kg, trans)
kg.save("knowledge_graph.json")            # persist

loaded_kg = KnowledgeGraph.load("knowledge_graph.json")
generator = TestsetGenerator(llm=generator_llm,
                             embedding_model=generator_embeddings,
                             knowledge_graph=loaded_kg)
```

> **COST & QUALITY CAUTIONS**
> Synthetic generation makes many LLM calls (transforms over every chunk, then synthesis per scenario). Start with a small `testset_size` and a few docs to gauge cost. Always human-review the result — RAGAS itself notes good synthetic data is hard, and you should select the queries you see fit for your final set.

7 Workflow Diagrams

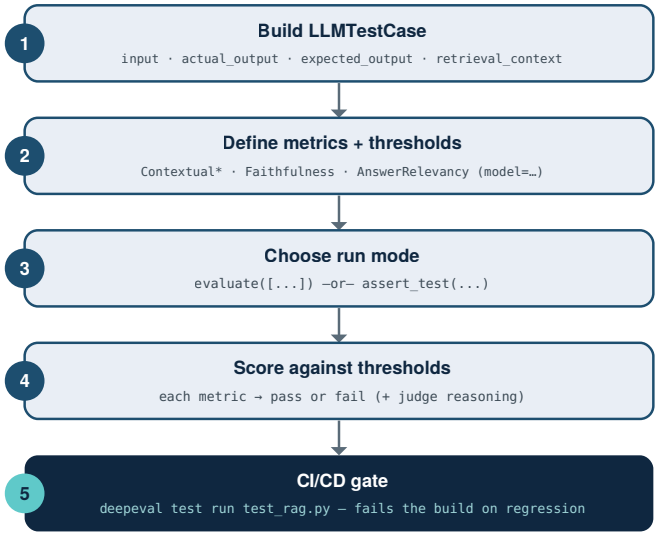
The step-by-step flow for each approach. Colour indicates which tool owns the step: **teal = RAGAS**, **navy = DeepEval**, **amber = handoff glue you write**.

7.1 Evaluating with RAGAS



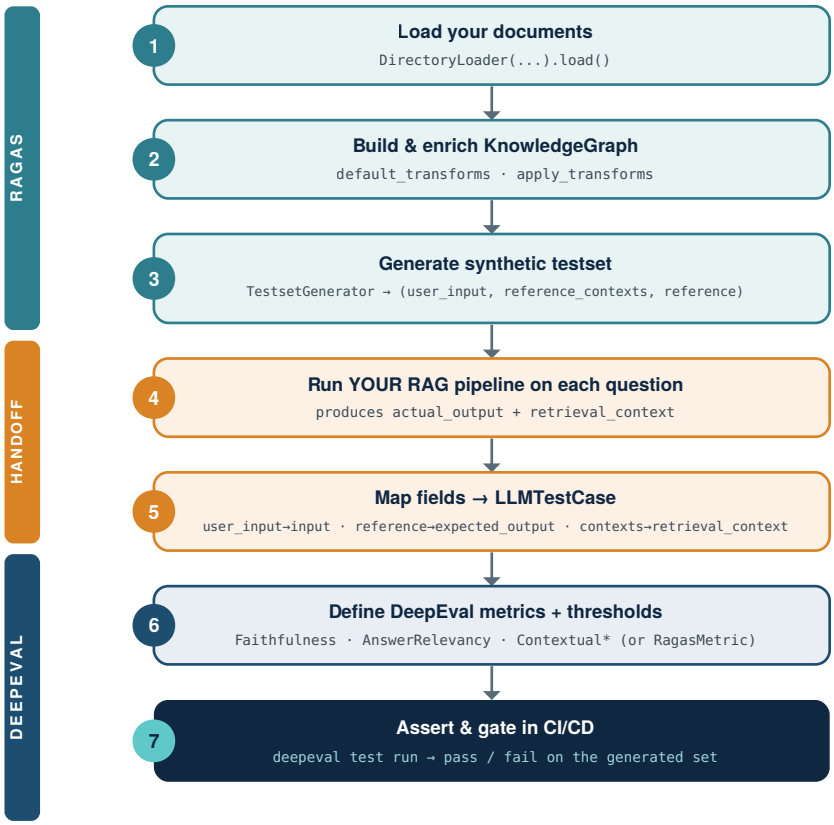
Measurement-oriented: one judge for the whole run, output is a scored table you inspect.

7.2 Evaluating with DeepEval



Gating-oriented: thresholds per metric, output is pass/fail suitable for a pipeline.

7.3 Combined — RAGAS generates, DeepEval evaluates



RAGAS bootstraps the test set from your docs (steps 1–3); you run your pipeline and remap field names (steps 4–5, the glue); DeepEval scores and gates (steps 6–7).

THE HANDOFF IS THE PART TO GET RIGHT

RAGAS emits `user_input / reference_contexts / reference`; DeepEval expects `input / retrieval_context / actual_output / expected_output`. Steps 4–5 are where you run your own RAG pipeline over the generated questions to obtain `actual_output` and the real `retrieval_context`, then rename the RAGAS fields to match. That glue code is yours to write — neither library does it automatically.